

Lecture – 12

Harimohan Khatri

1

In this lecture...

- i. Image Compression and its need
- ii. Lossy, Lossless Compression
- iii. Issues of Compression
- iv. Data Redundancy and Coding Redundancy
- v. A generic Model of Compression
- vi. Element of Information Theory (self information, entropy)

2

Image Compression

- The term data compression refers to the process of reducing the amount of data required to represent a given quantity of information.
- Data are the means by which information is conveyed.
- Because various amounts of data can be used to represent the same amount of information, representations that contain irrelevant or repeated information are said to contain redundant data.

3

Need of Image Compression

- **Storage Efficiency:** Compression reduces image file size, allowing more images to be stored in less space.
- **Faster Transmission:** Smaller image files enable quicker uploads, downloads, and data transfers.
- **Efficient Processing:** Compressed images load and process faster in applications and systems.
- **Cost Reduction:** Less storage and bandwidth usage leads to lower operational and infrastructure costs.
- **Quality Retention:** Compression can significantly reduce size while preserving acceptable visual quality.

4

Lossy and Lossless Compression

Lossless Compression

- Reduces file size without any loss of original image data that means the image can be perfectly reconstructed.
- Use Cases: Medical imaging, technical drawings, archival purposes

Lossy Compression

- Reduces file size by permanently removing some image data, leading to slight or noticeable quality loss.
- Use Cases: Photography, web graphics, streaming where small size and speed are more critical than perfect accuracy

5

Major Issues of Image Compression

- **Loss of Quality in Lossy Compression** - Compression may degrade image quality, especially at high compression ratios, leading to visible artifacts like blurring or blocking.
- **Compression Artifacts** - Lossy formats (e.g., JPEG) can introduce visual distortions such as ringing, blockiness, and color banding.
- **Computational Complexity** - Advanced compression algorithms (like JPEG2000 or HEVC) can be computationally intensive and slower in encoding/decoding.
- **Data Irrecoverability (in Lossy)** - Once lossy compression is applied, the discarded data cannot be restored, which is problematic in applications needing high fidelity.
- **Format Compatibility** - Not all software or systems support every compression format, leading to compatibility or interoperability issues.
- **Compression Ratio vs. Quality Trade-off** - Balancing file size and image quality is challenging—too much compression reduces usefulness, too little doesn't save enough space.

6

Redundancy in Image Data

- If we let b and b' denote the number of bits needed in two representations of the same information, the relative data redundancy, R , of the representation with b bits is

$$R = 1 - \frac{1}{C} \text{ where } C \text{ is compression ratio, is defined as } C = \frac{b}{b'}$$

7

Redundancy in Image Data – Coding Redundancy

- Occurs when symbols (e.g., pixel intensity values) are encoded using more bits than necessary.
- Using fixed-length codes (e.g., 8 bits for all pixel values from 0 to 255), even though some pixel values occur more frequently than others.
- Variable Length Coding addresses this by assigning shorter codes to frequent values and longer codes to rare ones, reducing average bits per pixel.
- Assume a discrete random variable r_k in the interval $[0,1]$ represents the gray levels of an image and that each r_k occurs with probability $p_r(r_k)$:

$$p_r(r_k) = \frac{n_k}{n} \quad k = 0, 1, \dots, L - 1$$

$$L_{avg} = \sum_{k=0}^{L-1} l(r_k) p_r(r_k)$$

8

Redundancy in Image Data – Coding Redundancy

r_k	$p_r(r_k)$	Code 1	$l_1(r_k)$	Code 2	$l_2(r_k)$
r_0	0.19	000	3	11	2
r_1	0.16	001	3	01	2
r_2	0.25	010	3	10	2
r_3	0.21	011	3	001	3
r_4	0.08	100	3	0001	4
r_5	0.06	101	3	00001	5
r_6	0.03	110	3	000101	6
r_7	0.02	111	3	000000	6

$$\text{Compression Ratio (C)} = \frac{3}{2.7} = 1.11$$

$$\text{Relative Redundancy (R)} = 1 - \frac{1}{C} = 1 - \frac{1}{1.11} = 0.099$$

$$\begin{aligned}
 L_{avg}(\text{code1}) &= \sum_{k=0}^{L-1} l(r_k) p_r(r_k) \\
 &= 3 * 0.19 + 3 * 0.16 \\
 &\quad + 3 * 0.25 + 3 * 0.21 \\
 &\quad + 3 * 0.08 + 3 * 0.06 \\
 &\quad + 3 * 0.03 + 3 * 0.02 \\
 &= 3 \text{ bits}
 \end{aligned}$$

$$\begin{aligned}
 L_{avg}(\text{code2}) &= \sum_{k=0}^{L-1} l(r_k) p_r(r_k) \\
 &= 2 * 0.19 + 2 * 0.16 \\
 &\quad + 2 * 0.25 + 3 * 0.21 \\
 &\quad + 4 * 0.08 + 5 * 0.06 \\
 &\quad + 6 * 0.03 + 6 * 0.02 \\
 &= 2.7 \text{ bits}
 \end{aligned}$$

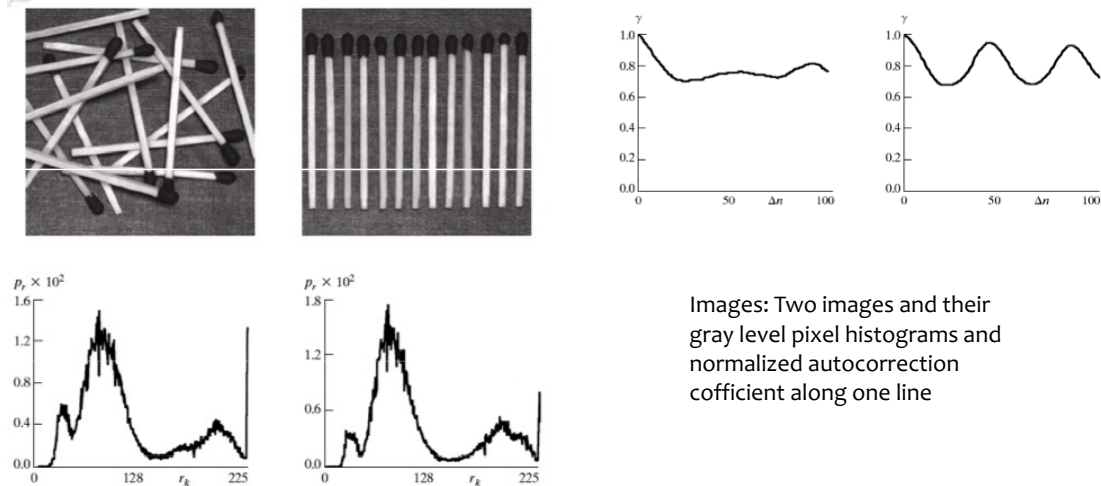
9

Redundancy in Image Data – Interpixel Redundancy

- In most images, pixels are correlated spatially (in both x and y) – Spatial Redundancy and in time (when the image is part of a video sequence) – Temporal Redundancy.
- Much of its visual contribution is redundant in the sense that it can be inferred from its neighbors.
- To reduce the redundancy associated with spatially and temporally correlated pixels, a 2-D intensity array must be transformed into a more efficient but usually “non-visual” representation.
- For example, run-lengths or the differences between adjacent pixels can be used. Transformations of this type are called mappings.

10

Redundancy in Image Data – Interpixel Redundancy



Images: Two images and their gray level pixel histograms and normalized autocorrelation coefficient along one line

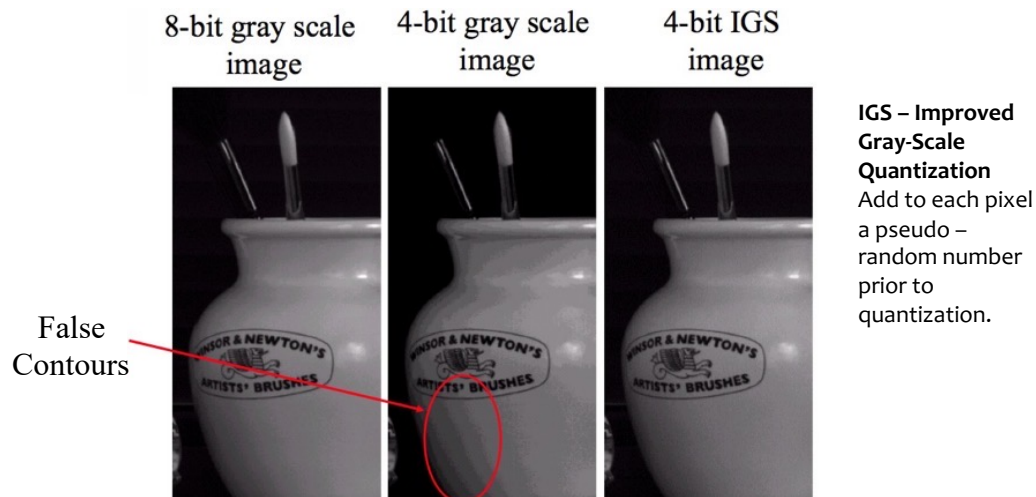
11

Redundancy in Image Data – Psychovisual Redundancy

- The human eye is less sensitive to high-frequency details, subtle color differences, or fine textures.
- It refers to the information in images that is not perceived by the human visual system, and can therefore be removed without noticeably affecting quality.
- The process of removing irrelevant information is known as quantization which generally means the mapping of a broad range of input values to a limited number of output values. Because information is lost, quantization is an irreversible operation.
- JPEG compression uses chroma subsampling because the eye is more sensitive to luminance (brightness) than chrominance (color).

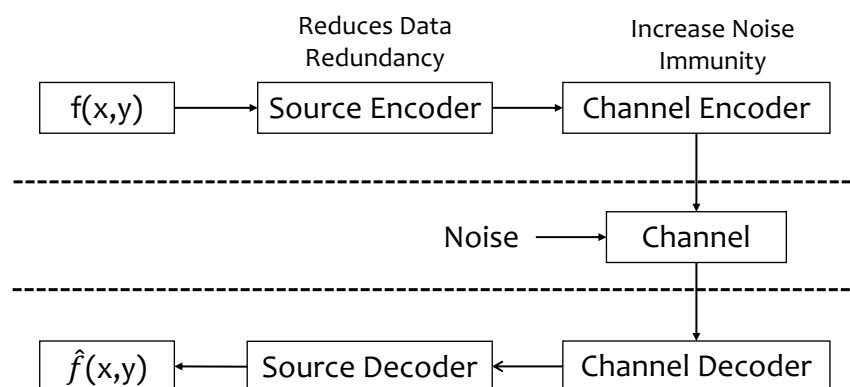
12

Redundancy in Image Data – Psychovisual Redundancy



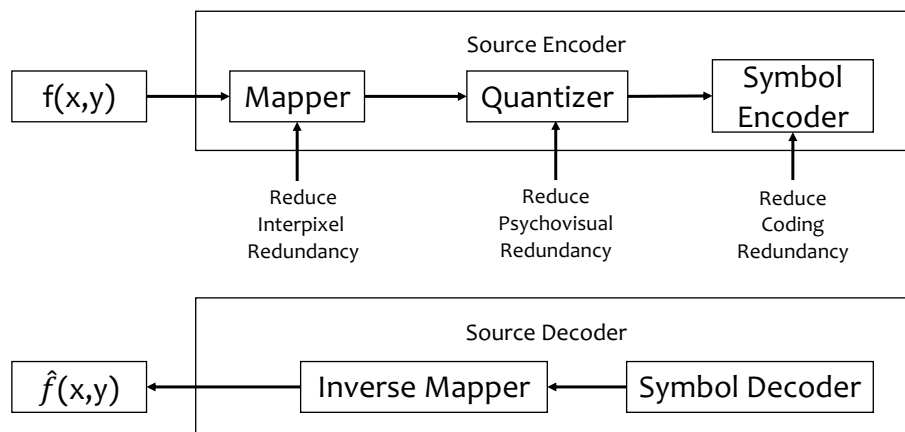
13

A Generic Model of Compression



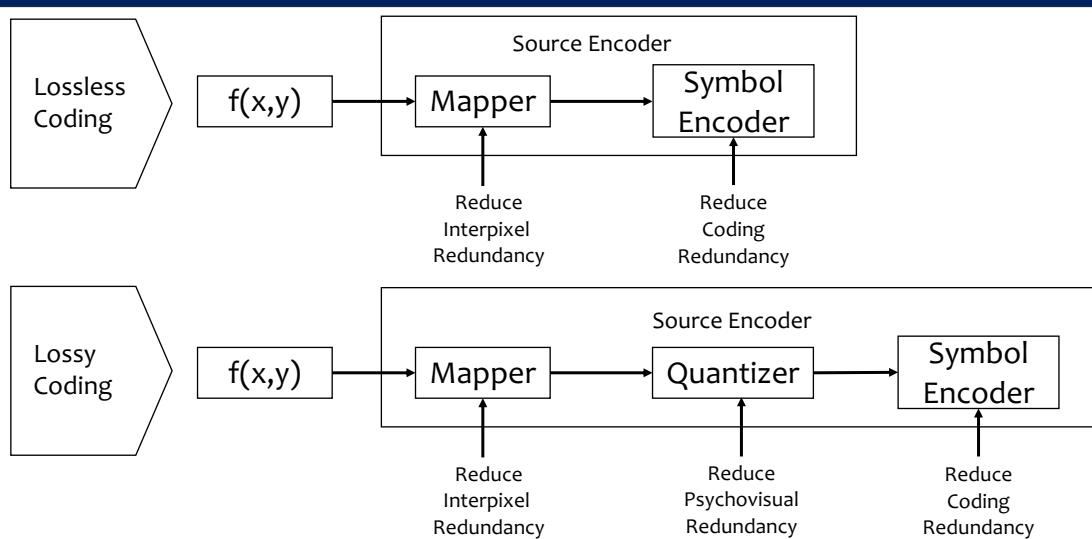
14

A Generic Model of Compression



15

A Generic Model of Compression



16

Element of Information Theory

- How few bits are actually needed to represent the information in an image? That is, is there a minimum amount of data that is sufficient to describe an image without losing information?
- Information theory provides the mathematical framework to answer this and related questions. Its fundamental premise is that the generation of information can be modeled as a probabilistic process which can be measured in a manner that agrees with intuition.
- A random event E with probability $P(E)$ is said to contain $I(E) = \log \frac{1}{P(E)} = -\log P(E)$ units of information.

17

Element of Information Theory

- we can use the histogram of the observed image to estimate the symbol probabilities of the source. Then, the intensity source's entropy becomes

$$\tilde{H} = - \sum_{k=0}^{L-1} p_r(r_k) \log_2 p_r(r_k)$$

18

Lecture – 13

Harimohan Khatri

19

In this lecture...

Types of Compression Techniques

- Entropy Encoding
 - i. Huffman Encoding (Coding Redundancy)
 - ii. Run-length Encoding (Interpixel Redundancy)
 - iii. LZW coding

20

Huffman Coding for Coding Redundancy

- Huffman Coding is a lossless data compression technique.
- The idea is to assign variable length codes to input characters, lengths of the assigned codes are based on the frequency of corresponding characters.
- The most frequent character gets the smallest code and the least frequent character gets the largest code.

21

Huffman Coding for Coding Redundancy

Question – 01: Perform Huffman Coding for given image data calculate L_{avg} , Entropy and Efficiency of encoding.

r	0	1	2	3	4	5	6	7
P(r)	0.02	0.05	0.12	0.1	0.3	0.15	0.03	0.23

Solution:

Step – 01: Arrange the intensity level (symbols) on descending order of probability of occurrence.

r	S4	S7	S5	S2	S3	S1	S6	S0
P(r)	0.3	0.23	0.15	0.12	0.1	0.05	0.03	0.02

22

Huffman Coding for Coding Redundancy

Step – 02: Source Reduction

r	P(r)	1	2	3	4	5	6
S4	0.3	0.3	0.3	0.3	0.3	→0.43	→ 0.57
S7	0.23	0.23	0.23	0.23	→0.27	0.3 ←	0.43
S5	0.15	0.15	0.15	→ 0.2	0.23 ←	0.27 ←	
S2	0.12	0.12	0.12	0.15 ←	0.2 ←		
S3	0.1	0.1	0.1 ←	0.12 ←			
S1	0.05	0.05 ←	→ 0.1 ←				
S6	0.03 ←	→ 0.05 ←					
S0	0.02 ←						

23

Huffman Coding for Coding Redundancy

Step – 03: Code Assignment

r	P(r)	1	2	3	4	5	6
S4	0.3 00	0.3 00	0.3 00	0.3 00	0.3 00	→ 0.43 1	→0.57 0
S7	0.23 10	0.23 10	0.23 10	0.23 10	→ 0.27 01	0.3 00 ←	0.43 1
S5	0.15 010	0.15 010	0.15 010	→ 0.2 11	0.23 10 ←	0.27 01 ←	
S2	0.12 011	0.12 011	0.12 011	0.15 010 ←	0.2 11 ←		
S3	0.1 110	0.1 110	0.1 110 ←	0.12 011 ←			
S1	0.05 1110	0.05 1110 ←	→ 0.1 111 ←				
S6	0.03 11110 ←	→ 0.05 1111 ←					
S0	0.02 11111 ←						

24

Huffman Coding for Coding Redundancy

Step – 04: Calculate L_{avg} , Entropy and Efficiency of encoding.

$$\begin{aligned} \text{Average Code Length } L_{avg} &= \sum_{k=0}^{L-1} l(r_k) p_r(r_k) \\ &= 0.3*2 + 0.23*2 + 0.15*3 + 0.12*3 + 0.1*3 + 0.05*4 + 0.03*5 + 0.02*5 \\ &= 2.62 \end{aligned}$$

$$\begin{aligned} \text{Entropy } \tilde{H} &= - \sum_{k=0}^{L-1} p_r(r_k) \log_2 p_r(r_k) \\ &= -(0.3*\log_2(0.3) + 0.23*\log_2(0.23) + 0.15*\log_2(0.15) + \\ &\quad 0.12*\log_2(0.12) + 0.1*\log_2(0.1) + 0.05*\log_2(0.05) + 0.03*\log_2(0.03) + \\ &\quad 0.02*\log_2(0.02)) \\ &= 2.48 \end{aligned}$$

$$\text{Efficiency} = \frac{\text{Entropy}}{L_{avg}} * 100 = \frac{2.48}{2.62} * 100 = 94.65$$

25

Huffman Coding for Coding Redundancy

Question – 02: Perform Huffman Coding for given image data calculate L_{avg} , Entropy and Efficiency of encoding.

Gray Levels	0	1	2	3	4	5	6	7
No of Pixels	42	124	166	252	512	711	313	54

26

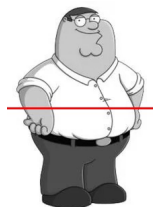
Run Length Coding for Interpixel Redundancy

- There may be identical pixel values in consecutive runs.
- Instead of storing each pixel value individually, RLC stores:
 - The value of the pixel
 - The length of the run (i.e., how many times it repeats consecutively)
- Example:
 - Original pixel row: [100, 100, 100, 50, 50, 200, 200, 200, 200]
 - RLC output: [(100, 3), (50, 2), (200, 4)]
- Efficient for images with large areas of uniform intensity (e.g., text, cartoons).
- Reduces interpixel redundancy, where neighboring pixels are the same.
- Not effective on noisy or complex images with high pixel variation.

27

Run Length Coding for Interpixel Redundancy

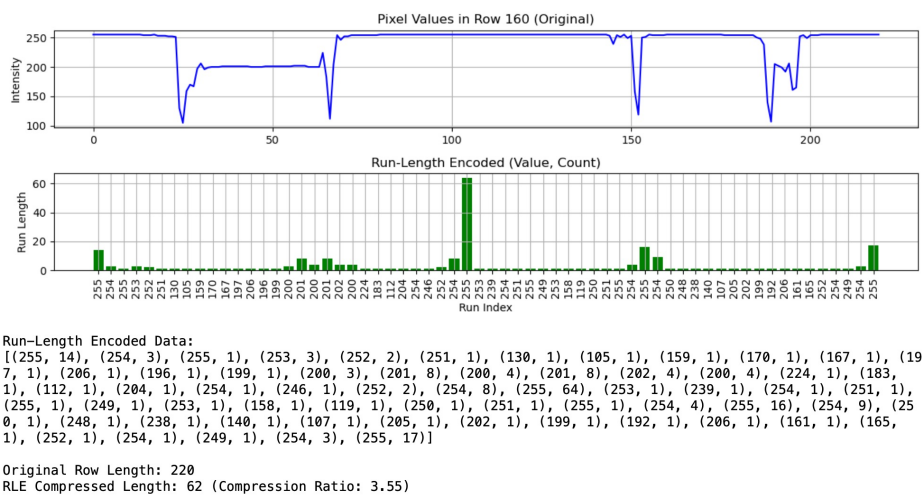
- To demonstrate Run Length Coding in grayscale image we have used a cartoon image sample of resolution 220 x 320.
- On which 160th row is scanned and its Run Length coding process is demonstrated below:



```
def run_length_encoding(row):
    rle = []
    prev_pixel = row[0]
    count = 1
    for pixel in row[1:]:
        if pixel == prev_pixel:
            count += 1
        else:
            rle.append((prev_pixel, count))
            prev_pixel = pixel
            count = 1
    rle.append((prev_pixel, count))
    return rle
```

28

Run Length Coding for Interpixel Redundancy



29

Lempel–Ziv–Welch (LZW) Coding

- Lempel-Ziv-Welch (LZW) coding, focuses on spatial redundancy, assigns fixed-length code words to variable length sequences of source symbols.
- In the beginning of the coding process, a dictionary containing the source symbols to be coded is constructed. For 8-bit monochrome images, the first 256 words of the dictionary are assigned to intensities 0, 1, 2, ..., 255.
- As the encoder sequentially examines image pixels, intensity sequences that are not in the dictionary are placed in algorithmically determined (e.g., the next unused) locations.

Dictionary Location	Entry
0	0
1	1
⋮	⋮
255	255
256	—
⋮	⋮
511	—

Entry upto 9-bit

30

Lempel–Ziv–Welch (LZW) Coding

Steps to perform LZW coding:

- The image is encoded by processing its pixels in a left-to-right, top-to-bottom manner.
- The successive intensity value is concatenated with a variable – called the “currently recognized sequence”.
- The dictionary is searched for each concatenated sequence and if found, is replaced by the newly concatenated and recognized sequence. No output codes are generated, nor is the dictionary altered.
- If the concatenated sequence is not found, the address of the currently recognized sequence is output as the next encoded value, the concatenated but unrecognized sequence is added to the dictionary, and the currently recognized sequence is initialized to the current pixel value.

31

Consider the following 8-bit image:

39 39 126 126
39 39 126 126
39 39 126 126
39 39 126 126

Currently Recognized Sequence	Pixel Being Processed	Encoded Output	Dictionary Location (Code Word)	Dictionary Entry
	39			
39	39	39	256	39-39
39	126	39	257	39-126
126	126	126	258	126-126
126	39	126	259	126-39
39	39			
39-39	126	256	260	39-39-126
126	126			
126-126	39	258	261	126-126-39
39	39			
39-39	126			
39-39-126	126	260	262	39-39-126-126
126	39			
126-39	39	259	263	126-39-39
39	126			
39-126	126	257	264	39-126-126
126		126		

16*8 bits

10*9 bits

32

Lempel–Ziv–Welch (LZW) Coding

Perform LZW coding for given 5 bit image calculate the efficiency.

12	12	12	12
12	12	12	12
26	26	26	26
26	26	26	26

33

Discussions and Classwork

34

Lecture – 14

Harimohan Khatri

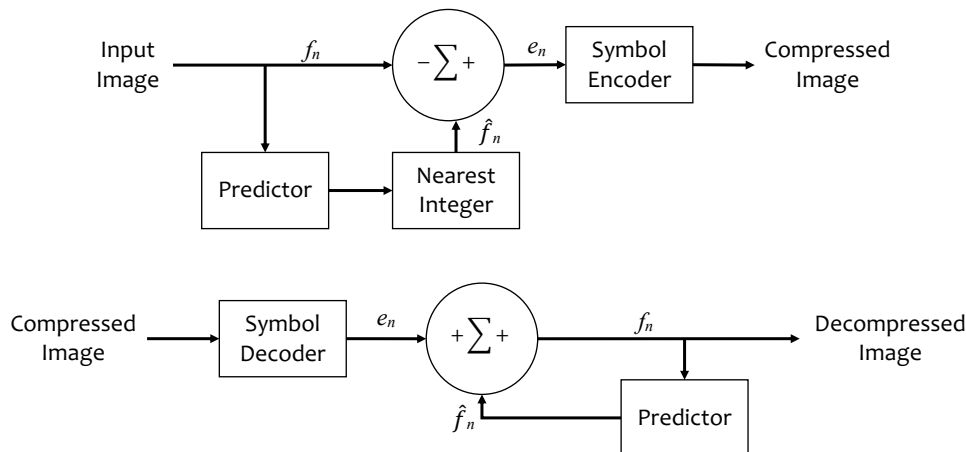
35

In this lecture...

- Types of Compression Techniques
 - Predictive Coding
 - Transform Coding

36

Lossless Predictive Coding



37

Lossless Predictive Coding – Encoder

- **Input Image:** The input image is represented as f_n
- **Predictor:** estimates the value of the current pixel $\hat{f}_n$ based on previously processed neighboring pixels. This step leverages spatial redundancy in the image.
- **Difference Calculation:** Prediction error $e_n = f_n - \hat{f}_n$
- **Symbol Encoder:** The prediction error e_n is then passed to a symbol encoder, which compresses e_n into a more compact form (e.g., using Huffman coding). The output is the compressed image data.

In many cases, the prediction is formed as a linear combination of m previous samples. $\hat{f}_n = \text{round} \left[\sum_{i=1}^m \alpha_i f_{n-i} \right]$

where m is the order of the linear predictor, round is a function used to denote the rounding or nearest integer operation, and the α_i for $i = 1, 2, \dots, m$ are prediction coefficients.

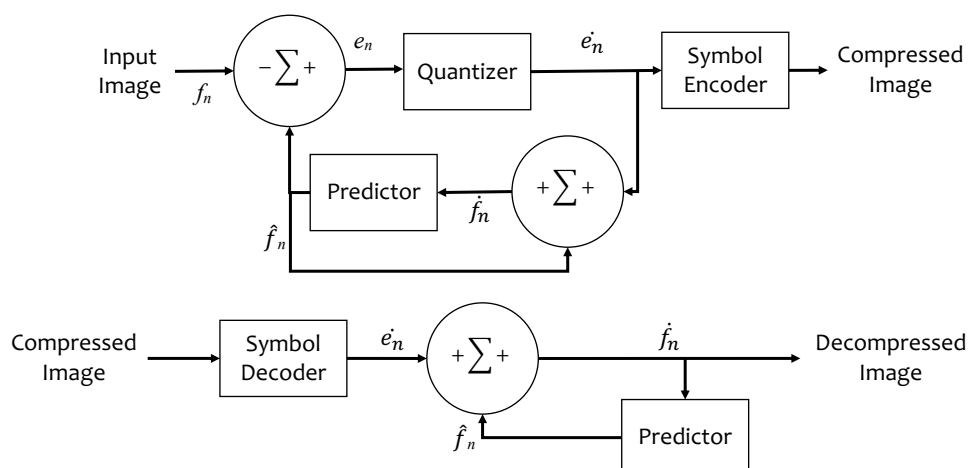
38

Lossless Predictive Coding – Decoder

- **Compressed Image:** compressed image data is the output from the symbol encoder.
- **Symbol Decoder:** reverses the encoding process, reconstructing the prediction error e_n from the compressed data.
- **Predictor:** It generates the predicted pixel value $\hat{f}_n$ based on previously decoded pixels.
- **Reconstruction:** The prediction error e_n is added to the predicted value $\hat{f}_n$, reconstructing the original pixel value f_n which is decompressed image output.

39

Lossy Predictive Coding



40

Lossy Predictive Coding – Encoder

- **Input Image:** Input image is represented by f_n
- **Predictor:** Generates an estimated pixel value $\hat{f}_n$ based on previously processed neighboring pixels, exploiting spatial redundancy.
- **Difference Calculation:** Prediction Error $e_n = f_n - \hat{f}_n$
- **Quantizer:** e_n is passed to a quantizer, which reduces the precision of e_n by mapping it to a discrete set of values. This step introduces lossy compression, as some information is discarded, but it significantly reduces the data size.
- **Symbol Encoder:** Quantization error $\hat{e}_n$ is then encoded by a symbol encoder (e.g., using Huffman coding) into a compressed format, producing the compressed image.

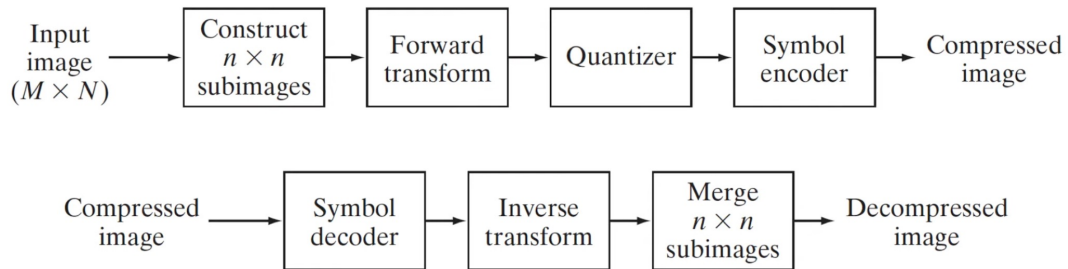
41

Lossy Predictive Coding – Decoder

- **Compressed Image:** compressed image data is the output from the symbol encoder.
- **Symbol Decoder:** It reverses the encoding process, reconstructing the quantized prediction error $\hat{e}_n$.
- **Predictor:** Generates the predicted pixel value $\hat{f}_n$ based on previously decoded pixels.
- **Reconstruction:** The quantized error $\hat{e}_n$ is added to the predicted value $\hat{f}_n$ reconstructing an approximation of the original pixel value f_n which is decompressed image output.

42

Transform Coding – Block Diagram



<https://www.youtube.com/watch?v=FTv61OEPLtY>

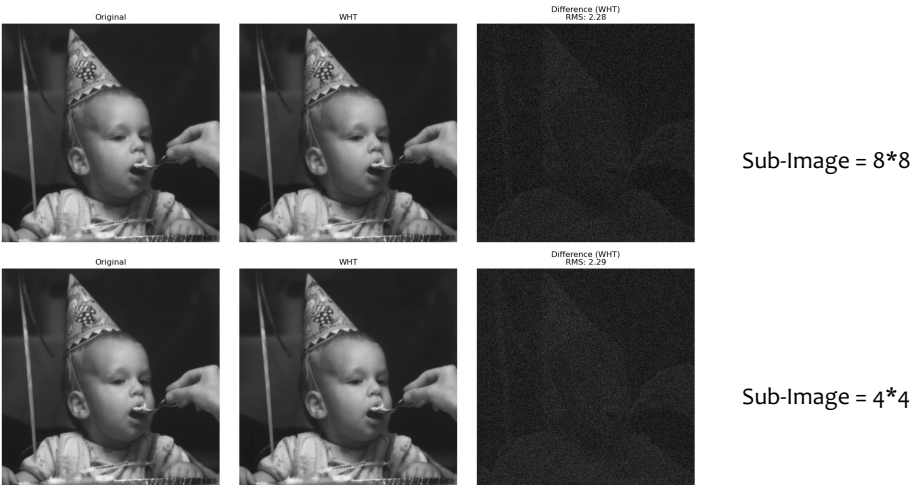
43

Transform Coding

- There are mainly three parameters that affect the performance of transform coding
 - Type of Transformation
 - Size of Sub-Image
 - Quantization Algorithm
- To demonstrate these factors some examples are given below;
- The uniform quantization function is used with $q_steps = 10$
 - For example: DCT coefficient = 57.6
 - With $q_step = 1 \rightarrow \text{round}(57.6 / 1) = 58 \rightarrow 58 \times 1 = 58$ (little loss)
 - With $q_step = 10 \rightarrow \text{round}(57.6 / 10) = 6 \rightarrow 6 \times 10 = 60$ (more loss)

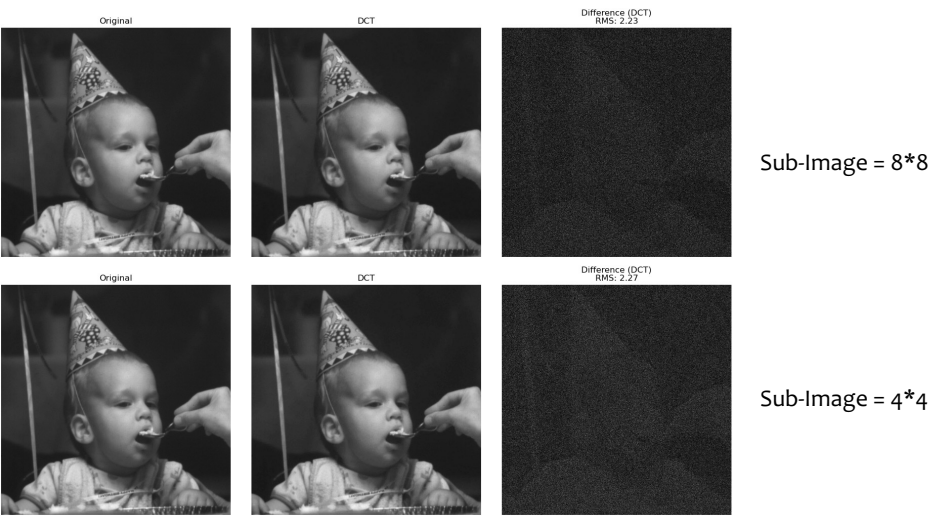
44

Transform Coding – Hadamard Transform



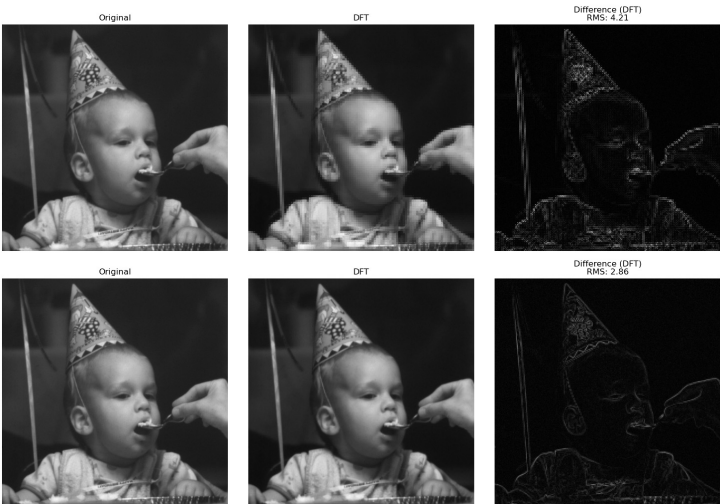
45

Transform Coding – Discrete Cosine Transform



46

Transform Coding – Discrete Fourier Transform



Sub-Image = 8*8

Sub-Image = 4*4

Discussions and Classwork